

Clustering Images

Let's look at an example of clustering images based on their color content. The file *sunsets.zip* contains 100 images downloaded from Flickr using the tag “sunset” or “sunsets.” For this example, we will use a color histogram of each image as feature vector. This is a bit crude and simple but good enough for illustrating what hierarchical clustering does. Try running the following code in a folder containing the sunset images:

```
import os
import hcluster

# create a list of images
path = 'flickr-sunsets/'
imlist = [os.path.join(path,f) for f in os.listdir(path) if f.endswith('.jpg')]

# extract feature vector (8 bins per color channel)
features = zeros([len(imlist), 512])
for i,f in enumerate(imlist):
    im = array(Image.open(f))

    # multi-dimensional histogram
    h,edges = histogramdd(im.reshape(-1,3),8,normed=True,
                          range=[(0,255),(0,255),(0,255)])
    features[i] = h.flatten()

tree = hcluster.hcluster(features)
```

Here we take the R, G, and B color channels as vectors and feed them into NumPy's `histogramdd()`, which computes multi-dimensional histograms (in this case three dimensions). We chose 8 bins in each color dimension ($8 \times 8 \times 8$), which, after flattening, gives 512 bins in the feature vector. We use the “normed=True” option to normalize the histograms in case the images are of different size and set the range to 0 . . . 255 for each color channel. The use of `reshape()` with one dimension set to -1 will automatically determine the correct size, and thereby create an input array to the histogram computation consisting of the RGB color values as rows.

To visualize the cluster tree, we can draw a dendrogram. A *dendrogram* is a diagram that shows the tree layout. This often gives useful information on how good a given descriptor vector is and what is considered similar in a particular case. Add the following code to *hcluster.py*:

```
from PIL import Image,ImageDraw

def draw_dendrogram(node,imlist,filename='clusters.jpg'):
    """ Draw a cluster dendrogram and save to a file. """

    # height and width
    rows = node.get_height()*20
    cols = 1200

    # scale factor for distances to fit image width
    s = float(cols-150)/node.get_depth()

    # create image and draw object
    im = Image.new('RGB',(cols,rows),(255,255,255))
```

```

draw = ImageDraw.Draw(im)

# initial line for start of tree
draw.line((0,rows/2,20,rows/2),fill=(0,0,0))

# draw the nodes recursively
node.draw(draw,20,(rows/2),s,imlist,im)
im.save(filename)
im.show()

```

Here the dendrogram drawing uses a `draw()` method for each node. Add this method to the `ClusterNode` class:

```

def draw(self,draw,x,y,s,imlist,im):
    """ Draw nodes recursively with image
        thumbnails for leaf nodes. """

    h1 = int(self.left.get_height()*20 / 2)
    h2 = int(self.right.get_height()*20 / 2)
    top = y-(h1+h2)
    bottom = y+(h1+h2)

    # vertical line to children
    draw.line((x,top+h1,x,bottom-h2),fill=(0,0,0))

    # horizontal lines
    ll = self.distance*s
    draw.line((x,top+h1,x+ll,top+h1),fill=(0,0,0))
    draw.line((x,bottom-h2,x+ll,bottom-h2),fill=(0,0,0))

    # draw left and right child nodes recursively
    self.left.draw(draw,x+ll,top+h1,s,imlist,im)
    self.right.draw(draw,x+ll,bottom-h2,s,imlist,im)

```

The leaf nodes have their own special method to draw thumbnails of the actual images. Add this to the `ClusterLeafNode` class:

```

def draw(self,draw,x,y,s,imlist,im):
    nodeim = Image.open(imlist[self.id])
    nodeim.thumbnail([20,20])
    ns = nodeim.size
    im.paste(nodeim,[int(x),int(y-ns[1]//2),int(x+ns[0]),int(y+ns[1]-ns[1]//2)])

```

The height of a dendrogram and the sub parts are determined by the distance values. These need to be scaled to fit inside the chosen image resolution. The nodes are drawn recursively with the coordinates passed down to the level below. Leaf nodes are drawn with small thumbnail images of 20×20 pixels. Two helper methods are used to get the height and width of the tree, `get_height()` and `get_depth()`.

The dendrogram is drawn like this:

```

hcluster.draw_dendrogram(tree,imlist,filename='sunset.pdf')

```

The cluster dendrogram for the sunset images is shown in Figure 6-5. As can be seen, images with similar color are close in the tree. Three example clusters are shown in

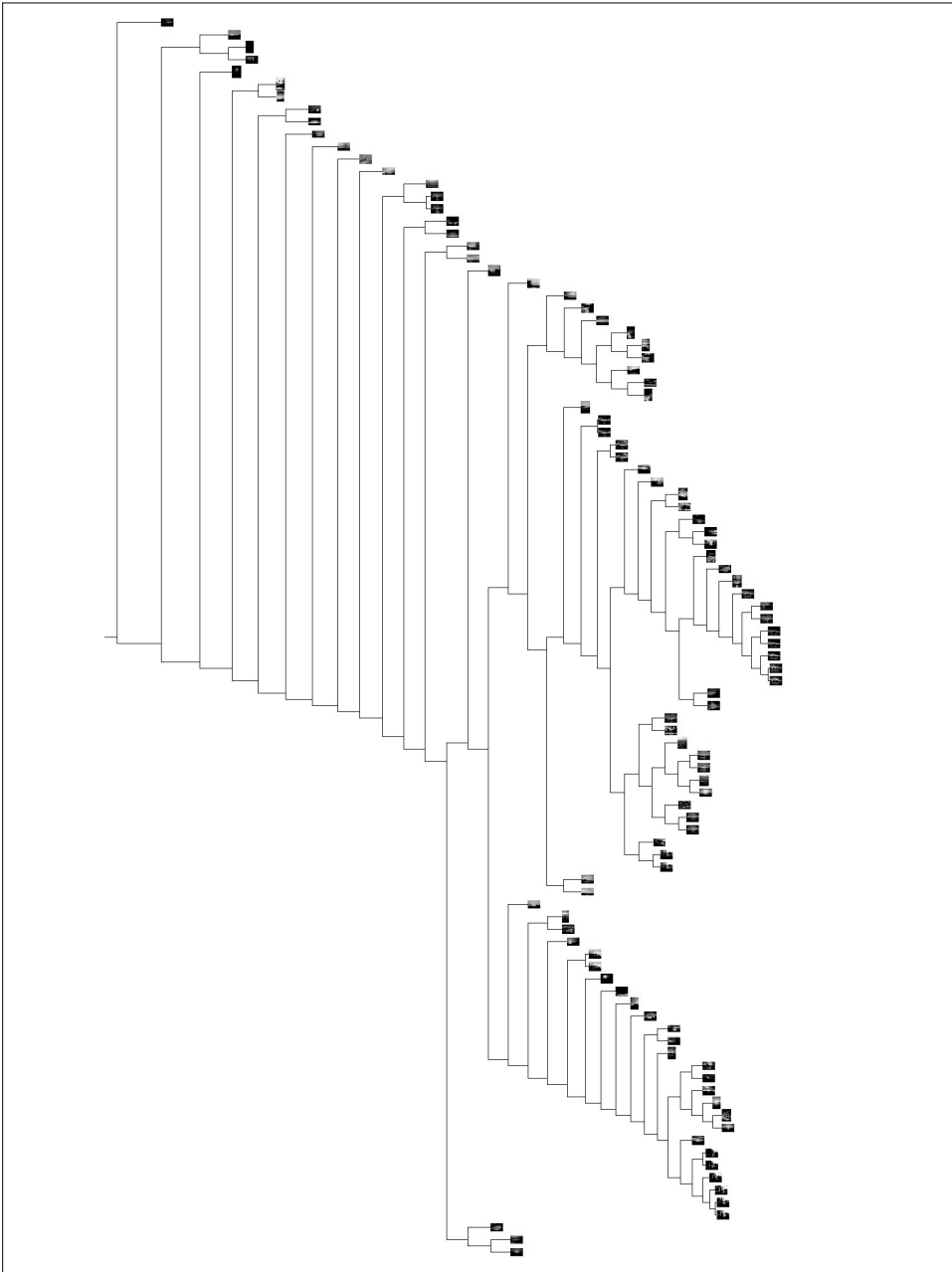


Figure 6-5. An example of hierarchical clustering of 100 images of sunsets using a 512 bin histogram in RGB coordinates as feature vector. Images close together in the tree have similar color distribution.



Figure 6-6. Example clusters from the 100 images of sunsets obtained with hierarchical clustering using a threshold set to 23% of the maximum node distance in the tree.

Figure 6-6. The clusters in this example are extracted as follows:

```
# visualize clusters with some (arbitrary) threshold
clusters = tree.extract_clusters(0.23*tree.distance)

# plot images for clusters with more than 3 elements
for c in clusters:
    elements = c.get_cluster_elements()
    nbr_elements = len(elements)
    if nbr_elements>3:
        figure()
        for p in range(minimum(nbr_elements,20)):
            subplot(4,5,p+1)
            im = array(Image.open(imlist[elements[p]]))
            imshow(im)
            axis('off')
        show()
```

As a final example, we can create a dendrogram for the font images:

```
tree = hcluster.hcluster(projected)
hcluster.draw_dendrogram(tree,imlist,filename='fonts.jpg')
```

where *projected* and *imlist* refer to the variables used in the *k*-means example in Section 6.1. The resulting font images dendrogram is shown in Figure 6-7.

6.3 Spectral Clustering

Spectral clustering methods are an interesting type of clustering algorithm that have a different approach compared to *k*-means and hierarchical clustering.

A *similarity matrix* (or *affinity matrix*, or sometimes *distance matrix*) for *n* elements (for example images) is an $n \times n$ matrix with pair-wise similarity scores. Spectral clustering

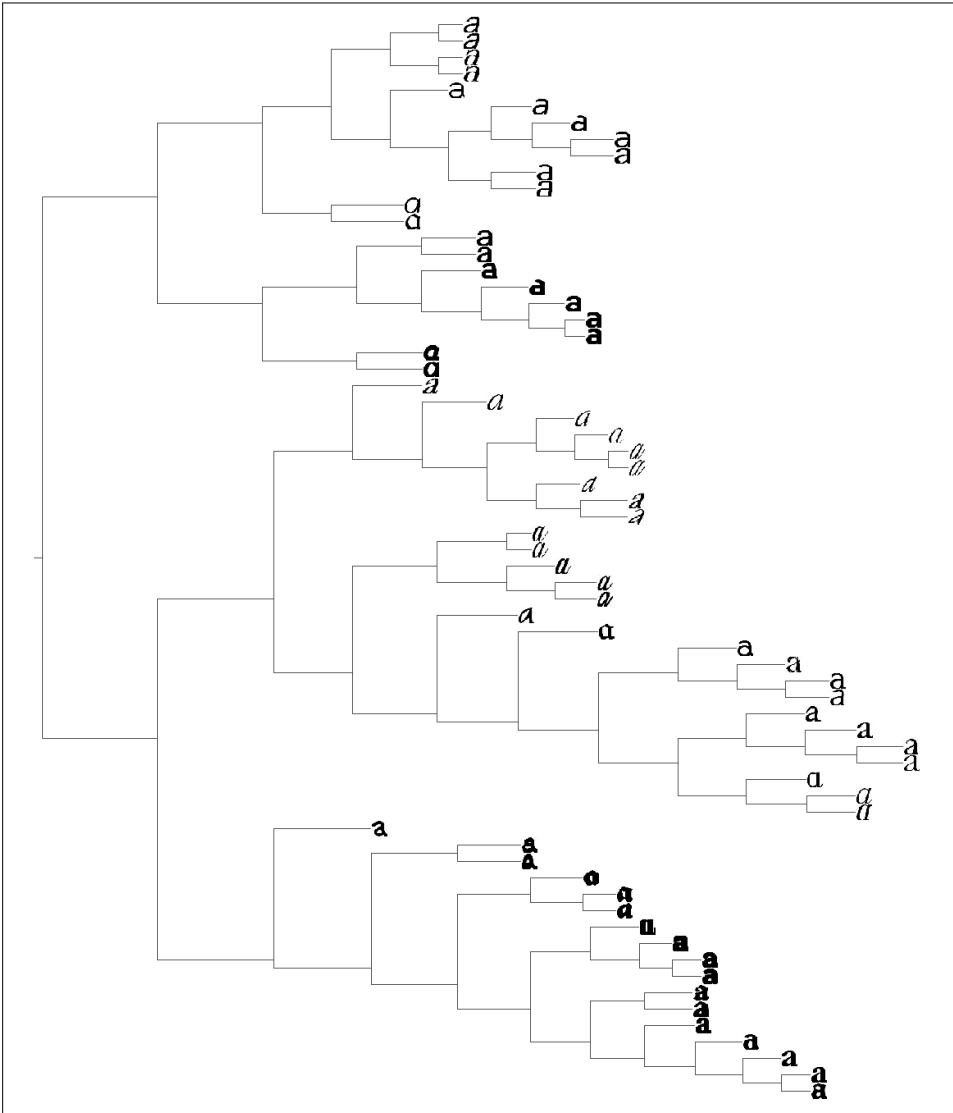


Figure 6-7. An example of hierarchical clustering of 66 selected font images using 40 principal components as feature vector.

gets its name from the use of the spectrum of a matrix constructed from a similarity matrix. The eigenvectors of this matrix are used for dimensionality reduction and then clustering.

One of the benefits of spectral clustering methods is that the only input needed is this matrix and it can be constructed from any measure of similarity you can think of. Methods like k -means and hierarchical clustering compute the mean of feature vectors, and this restricts the features (or descriptors) to vectors (in order to be able to compute

the mean). With spectral methods, there is no need to have feature vectors of any kind, just a notion of “distance” or “similarity.”

Here’s how it works. Given a $n \times n$ similarity matrix S with similarity scores s_{ij} , we can create a matrix, called the *Laplacian matrix*²,

$$L = I - D^{-1/2} S D^{-1/2},$$

where I is the identity matrix and D is the diagonal matrix containing the row sums of S , $D = \text{diag}(d_i)$, $d_i = \sum_j s_{ij}$. The matrix $D^{-1/2}$ used in the construction of the Laplacian matrix is then

$$D^{-1/2} = \begin{bmatrix} \frac{1}{\sqrt{d_1}} & & & \\ & \frac{1}{\sqrt{d_2}} & & \\ & & \ddots & \\ & & & \frac{1}{\sqrt{d_n}} \end{bmatrix}.$$

In order to make the presentation clearer, let’s use low values of s_{ij} for similar elements and require $s_{ij} \geq 0$ (the term distance matrix is perhaps more fitting in this case).

The clusters are found by computing the eigenvectors of L and using the k eigenvectors corresponding to the k largest eigenvalues to construct a set of feature vectors (remember that we may not have had any to start with!). Create a matrix with the k eigenvectors as columns. The rows will then be treated as new feature vectors (of length k). These new feature vectors can then be clustered, using, for example, k -means to produce the final clusters. In essence, what the algorithm does is to transform the original data into new feature vectors that can be more easily clustered (and in some cases using cluster algorithms that could not be used in the first place).

Enough about the theory; let’s see what it looks like in code when applied to a real example. Again, we take the font images used in the k -means example above (and introduced on page 14):

```
from scipy.cluster.vq import *

n = len(projected)

# compute distance matrix
S = array([[ sqrt(sum((projected[i]-projected[j])**2))
              for i in range(n) ] for j in range(n)], 'f')

# create Laplacian matrix
rowsum = sum(S,axis=0)
D = diag(1 / sqrt(rowsum))
I = identity(n)
L = I - dot(D,dot(S,D))
```

² Sometimes $L = D^{-1/2} S D^{-1/2}$ is used as the Laplacian matrix instead, but the choice doesn’t really matter since it only changes the eigenvalues, not the eigenvectors.

```

# compute eigenvectors of L
U,sigma,V = linalg.svd(L)

k = 5
# create feature vector from k first eigenvectors
# by stacking eigenvectors as columns
features = array(V[:k]).T

# k-means
features = whiten(features)
centroids,distortion = kmeans(features,k)
code,distance = vq(features,centroids)

# plot clusters
for c in range(k):
    ind = where(code==c)[0]
    figure()
    for i in range(minimum(len(ind),39)):
        im = Image.open(path+imlist[ind[i]])
        subplot(4,10,i+1)
        imshow(array(im))
        axis('equal')
        axis('off')
show()

```

In this case, we just create S using pair-wise Euclidean distances and compute a standard k -means clustering on the k eigenvectors ($k = 5$ in this particular case). Remember that the matrix V contains the eigenvectors sorted with respect to the eigenvalues. Finally, the clusters are plotted. Figure 6-8 shows the clusters for an example run (remember that the k -means step might give different results each run).

We can also try this on an example where we don't have any feature vectors or any strict definition of similarity. The geotagged Panoramio images on page 44 were linked based on how many matching local descriptors were found between them. The matrix

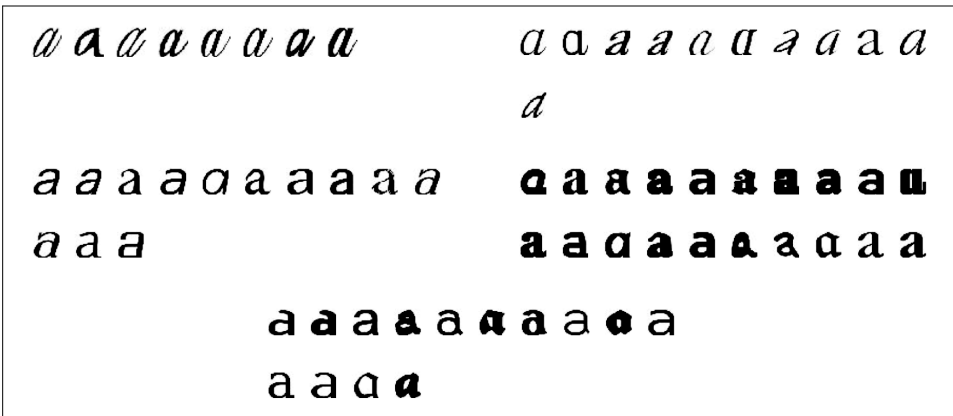


Figure 6-8. Spectral clustering of font images using the eigenvectors of the Laplacian matrix.

on page 48 is a similarity matrix with scores equal to the number of matching features (without any normalization). With *imlist* containing the filenames of the images and the similarity matrix saved to a file using NumPy's `savetxt()`, we only need to modify the first rows of the code above to

```
n = len(imlist)

# load the similarity matrix and reformat
S = loadtxt('panoramio_matches.txt')
S = 1 / (S + 1e-6)
```

where we invert the scores to have low values for similar images (so we don't have to modify the code above). We add a small number to avoid division with zero. The rest of the code you can leave as is.

Choosing *k* is a bit tricky in this case. Most people would consider there to be only two classes (the two sides of the White House) and then some junk images. With *k* = 2, you get something like Figure 6-9, with one large cluster of images of one side and the other cluster containing the other side plus all the junk images. Picking a larger value of *k*, like *k* = 10, gives several clusters with only one image (hopefully the junk images) and some real clusters. An example run is shown in Figure 6-10. In this case, there were only two actual clusters, each containing images of one side of the White House.

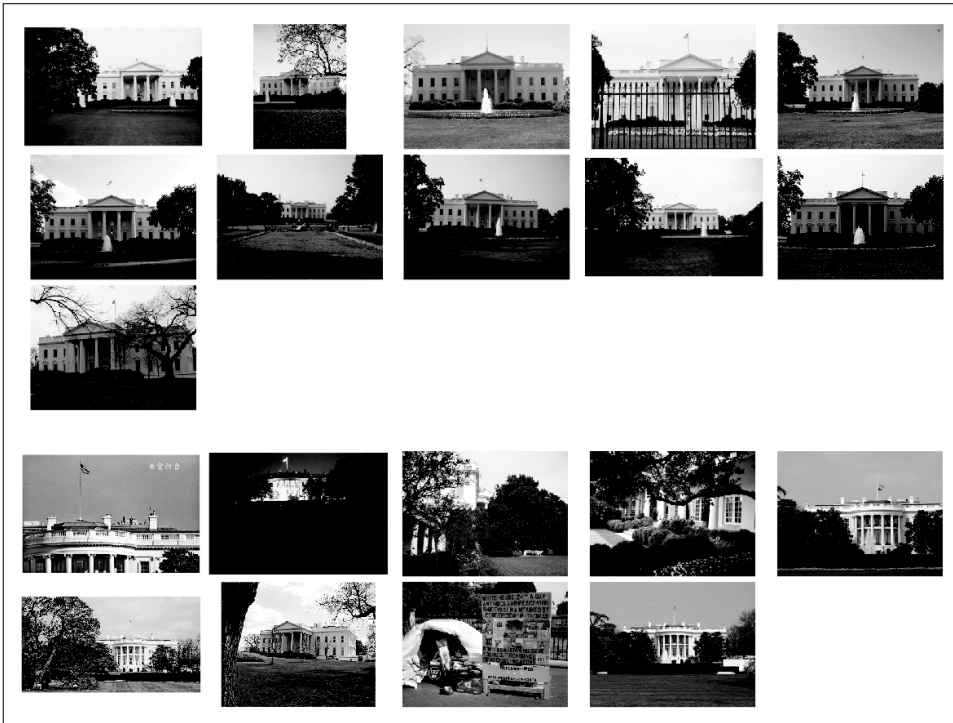


Figure 6-9. Spectral clustering of geotagged images of the White House with *k* = 2 and the similarity scores as the number of matching local descriptors.



Figure 6-10. Spectral clustering of geotagged images of the White House with $k = 10$ and the similarity scores as the number of matching local descriptors. Only the clusters with more than one image shown.

There are many different versions and alternatives to the algorithm presented here, each of them with its own idea of how to construct the matrix L and what to do with the eigenvectors. For further reading on spectral clustering and the details of some common algorithms, see for example the review paper [37].

Exercises

1. *Hierarchical k -means* is a clustering method that applies k -means recursively to the clusters to create a tree of incrementally refined clusters. In this case, each node in the tree will branch to k child nodes. Implement this and try it on the font images.
2. Using the hierarchical k -means from the previous exercise, make a tree visualization (similar to the dendrogram for hierarchical clustering) that shows the average image for each cluster node. Tip: You can take the average PCA coefficients feature vector and use the PCA basis to synthesize an image for each feature vector.
3. By modifying the class used for hierarchical clustering to include the number of images below the node, you have a simple and fast way of finding similar (tight) groups of a given size. Implement this small change and try it out on some real data. How does it perform?
4. Experiment with using single and complete linking for building the hierarchical cluster tree. How do the resulting clusters differ?
5. In some spectral clustering algorithms the matrix $D^{-1}S$ is used instead of L . Try replacing the Laplacian matrix with this and apply this on a few different data sets.
6. Download some image collections from Flickr searches with different tags. Extract the RGB histogram like you did for the sunset images. Cluster the images using one of the methods from this chapter. Can you separate the classes with the clusters?

Searching Images

This chapter shows how to use text mining techniques to search for images based on their visual content. The basic ideas of using visual words are presented and the details of a complete setup are explained and tested on an example image data set.

7.1 Content-Based Image Retrieval

Content-based image retrieval (CBIR) deals with the problem of retrieving visually similar images from a (large) database of images. This can be images with similar color, similar textures, or similar objects or scenes: basically any information contained in the images themselves.

For high-level queries, like finding similar objects, it is not feasible to do a full comparison (for example using feature matching) between a query image and all images in the database. It would simply take too much time to return any results if the database is large. In the last couple of years, researchers have successfully introduced techniques from the world of text mining for CBIR problems, making it possible to search millions of images for similar content.

Inspiration from Text Mining—The Vector Space Model

The *vector space model* is a model for representing and searching text documents. As we will see, it can be applied to essentially any kind of objects, including images. The name comes from the fact that text documents are represented with vectors that are histograms of the word frequencies in the text.¹ In other words, the vector will contain the number of occurrences of every word (at the position corresponding to that word) and zeros everywhere else. This model is also called a *bag-of-word representation*, since order and location of words is ignored.

¹ Often you see “term” used instead of “word”; the meaning is the same.

Documents are indexed by doing a word count to construct the document histogram vector $\mathbf{v}$, usually with common words like “the,” “and,” “is,” etc., ignored. These common words are called *stop words*. To compensate for document length, the vectors can be normalized to unit length by dividing with the total histogram sum. The individual components of the histogram vector are usually weighted according to the importance of each word. Usually, the importance of a word increases proportionally to how often it appears in the document, but decreases if the word is common in all documents in a data set (or “corpus”).

The most common weighting is *tf-idf weighting* (*term frequency–inverse document frequency*) where the *term frequency* of a word w in document d is

$$\text{tf}_{w,d} = \frac{n_w}{\sum_j n_j},$$

where n_w is the number of occurrences of w in d . To normalize, this is divided by the total number of occurrences of all words in the document.

The *inverse document frequency* is

$$\text{idf}_{w,d} = \log \frac{|D|}{|\{d: w \in d\}|},$$

where $|D|$ is the number of documents in the corpus D and the denominator the number of documents d in D containing w . Multiplying the two gives the tf-idf weight, which becomes one of the elements in $\mathbf{v}$. You can read more about tf-idf at <http://en.wikipedia.org/wiki/Tf-idf>.

This is really all we need at the moment. Let’s see how to carry this model over to indexing and searching images based on their visual content.

7.2 Visual Words

To apply text mining techniques to images, we first need to create the visual equivalent of words. This is usually done using local descriptors like the SIFT descriptor in Section 2.2. The idea is to quantize the descriptor space into a number of typical examples and assign each descriptor in the image to one of those examples. These typical examples are determined by analyzing a training set of images and can be considered as *visual words*. The set of all words is then a *visual vocabulary* (sometimes called a *visual codebook*). This vocabulary can be created specifically for a given problem or type of image or just to try to represent visual content in general.

The visual words are constructed using some clustering algorithm applied to the feature descriptors extracted from a (large) training set of images. The most common choice is k -means,² which is what we will use here. Visual words are nothing but a collection of vectors in the given feature descriptor space; in the case of k -means, they are the cluster centroids. Representing an image with a histogram of visual words is then called a *bag-of-visual-words* model.

² Or in the more advanced cases, hierarchical k -means.

Let's introduce an example data set and use that to illustrate the concept. The file *first1000.zip* contains the first 1000 images from the University of Kentucky object recognition benchmark set (also known as “ukbench”). The full set, reported benchmarks, and some supporting code can be found at <http://www.vis.uky.edu/~stewe/ukbench/>. The ukbench set contains many sets of four images, each of the same scene or object (stored consecutively so that 0 . . . 3, 4 . . . 7, etc., belong together). Figure 7-1 shows some examples from the data set. Appendix A has the details on the set and how to get the data.



Figure 7-1. Some examples of images from the ukbench (University of Kentucky object recognition benchmark) data set.

Creating a Vocabulary

To create a vocabulary of visual words we first need to extract descriptors. Here we will use the SIFT descriptor. Running the following lines of code, with *imlist*, as usual, containing the filenames of the images,

```
nbr_images = len(imlist)
featlist = [ imlist[i][:-3]+'sift' for i in range(nbr_images)]

for i in range(nbr_images):
    sift.process_image(imlist[i],featlist[i])
```

will give you descriptor files for each image. Create a file *vocabulary.py* and add the following code for a vocabulary class and a method for training a vocabulary on some training image data:

```
from scipy.cluster.vq import *
import vlfeat as sift

class Vocabulary(object):

    def __init__(self,name):
        self.name = name
        self.voc = []
        self.idf = []
        self.trainingdata = []
        self.nbr_words = 0

    def train(self,featurefiles,k=100,subsampling=10):
        """ Train a vocabulary from features in files listed
            in featurefiles using k-means with k number of words.
            Subsampling of training data can be used for speedup. """

        nbr_images = len(featurefiles)
        # read the features from file
        descr = []
        descr.append(sift.read_features_from_file(featurefiles[0])[1])
        descriptors = descr[0] #stack all features for k-means
        for i in arange(1,nbr_images):
            descr.append(sift.read_features_from_file(featurefiles[i])[1])
            descriptors = vstack((descriptors,descr[i]))

        # k-means: last number determines number of runs
        self.voc,distortion = kmeans(descriptors[:,::subsampling,:],k,1)
        self.nbr_words = self.voc.shape[0]

        # go through all training images and project on vocabulary
        imwords = zeros((nbr_images,self.nbr_words))
        for i in range( nbr_images ):
            imwords[i] = self.project(descr[i])

        nbr_occurences = sum( (imwords > 0)*1 ,axis=0)

        self.idf = log( (1.0*nbr_images) / (1.0*nbr_occurences+1) )
        self.trainingdata = featurefiles
```

```

def project(self, descriptors):
    """ Project descriptors on the vocabulary
        to create a histogram of words. """

    # histogram of image words
    imhist = zeros((self.nbr_words))
    words, distance = vq(descriptors, self.voc)
    for w in words:
        imhist[w] += 1

    return imhist

```

The class `Vocabulary` contains a vector of word cluster centers *voc* together with the *idf* values for each word. To train the vocabulary on some set of images, the method `train()` takes a list of `.sift` descriptor files and *k*, the desired number of words for the vocabulary. There is also an option of subsampling the training data for the *k*-means step, which will take a long time if too many features are used.

With the images and the feature files in a folder on your computer, the following code will create a vocabulary of length $k \approx 1000$ (again assuming that *imlist* contains a list of filenames for the images):

```

import pickle
import vocabulary

nbr_images = len(imlist)
featlist = [ imlist[i][:-3]+'sift' for i in range(nbr_images) ]

voc = vocabulary.Vocabulary('ukbenchtest')
voc.train(featlist, 1000, 10)

# saving vocabulary
with open('vocabulary.pkl', 'wb') as f:
    pickle.dump(voc, f)
print 'vocabulary is:', voc.name, voc.nbr_words

```

The last part also saves the entire vocabulary object for later use using the `pickle` module.

7.3 Indexing Images

Before we can start searching, we need to set up a database with the images and their visual word representations.

Setting Up the Database

To start indexing images, we first need to set up a database. Indexing images in this context means extracting descriptors from the images, converting them to visual words using a vocabulary, and storing the visual words and word histograms with information about which image they belong to. This will make it possible to query the database using an image and get the most similar images back as search result.

Table 7-1. A simple database schema for storing images and visual words.

imlist	imwords	imhistograms
rowid	imid	imid
filename	wordid	histogram
	vocname	vocname

Here we will use SQLite as database. SQLite is a database that stores everything in a single file and is very easy to set up and use. We are using it here since it is the easiest way to get started without having to go into database and server configurations and other details way outside the scope of this book. The Python version, `pysqlite`, is available from <http://code.google.com/p/pysqlite/> and also through many package repositories on Mac and Linux. SQLite uses the SQL query language so the transition should be easy if you want to use another database.

To get started, we need to create tables and indexes and an indexer class to write image data to the database. First, create a file *imagesearch.py* and add the following code:

```
import pickle
from pysqlite2 import dbapi2 as sqlite

class Indexer(object):

    def __init__(self,db,voc):
        """ Initialize with the name of the database
            and a vocabulary object. """

        self.con = sqlite.connect(db)
        self.voc = voc

    def __del__(self):
        self.con.close()

    def db_commit(self):
        self.con.commit()
```

First of all, we need Pickle for encoding and decoding these arrays to and from strings. SQLite is imported from the `pysqlite2` module (see Appendix A for installation details). The `Indexer` class connects to a database and stores a vocabulary object upon creation (where the `__init__()` method is called). The `__del__()` method makes sure to close the database connection and `db_commit()` writes the changes to the database file.

We only need a very simple database schema of three tables. The table `imlist` contains the filenames of all indexed images, and `imwords` contains a *word index* of the words, which vocabulary was used, and which images the words appear in. Finally, `imhistograms` contains the full word histograms for each image. We need those to compare images according to our vector space model. The schema is shown in Table 7-1.

The following method for the `Indexer` class creates the tables and some useful indexes to make searching faster:


```

def create_tables(self):
    """ Create the database tables. """

    self.con.execute('create table imlist(filename)')
    self.con.execute('create table imwords(imid,wordid,vocname)')
    self.con.execute('create table imhistograms(imid,histogram,vocname)')
    self.con.execute('create index im_idx on imlist(filename)')
    self.con.execute('create index wordid_idx on imwords(wordid)')
    self.con.execute('create index imid_idx on imwords(imid)')
    self.con.execute('create index imidhist_idx on imhistograms(imid)')
    self.db_commit()

```

Adding Images

With the database tables in place, we can start adding images to the index. To do this, we need a method `add_to_index()` for our `Indexer` class. Add this method to *imagesearch.py*:

```

def add_to_index(self, imname, descr):
    """ Take an image with feature descriptors,
        project on vocabulary and add to database. """

    if self.is_indexed(imname): return
    print 'indexing', imname

    # get the imid
    imid = self.get_id(imname)

    # get the words
    imwords = self.voc.project(descr)
    nbr_words = imwords.shape[0]

    # link each word to image
    for i in range(nbr_words):
        word = imwords[i]
        # wordid is the word number itself
        self.con.execute("insert into imwords(imid,wordid,vocname)
            values (?,?,?)", (imid,word,self.voc.name))

    # store word histogram for image
    # use pickle to encode NumPy arrays as strings
    self.con.execute("insert into imhistograms(imid,histogram,vocname)
        values (?,?,?)", (imid,pickle.dumps(imwords),self.voc.name))

```

This method takes the image filename and a NumPy array with the descriptors found in the image. The descriptors are projected on the vocabulary and inserted in `imwords` (word by word) and `imhistograms`. We used two helper functions, `is_indexed()`, which checks if the image has been indexed already, and `get_id()`, which gives the image id for an image filename. Add these to *imagesearch.py*:

```

def is_indexed(self, imname):
    """ Returns True if imname has been indexed. """

    im = self.con.execute("select rowid from imlist where
        filename='%s'" % imname).fetchone()
    return im != None

```

```

def get_id(self, imname):
    """ Get an entry id and add if not present. """

    cur = self.con.execute(
        "select rowid from imlist where filename='%s'" % imname)
    res=cur.fetchone()
    if res==None:
        cur = self.con.execute(
            "insert into imlist(filename) values ('%s')" % imname)
        return cur.lastrowid
    else:
        return res[0]

```

Did you notice that we used Pickle in `add_to_index()`? Databases like SQLite do not have a standard type for storing objects or arrays. Instead, we can create a string representation using Pickle's `dumps()` function and write the string to the database. Consequently, we need to un-pickle the string when reading from the database. More on that in the next section.

The following code example will go through the `ukbench` sample images and add them to our index. Here we assume that the lists *imlist* and *featlist* contain the filenames of the images and the descriptor files and that the vocabulary you trained earlier was pickled to a file *vocabulary.pkl*:

```

import pickle
import sift
import imagesearch

nbr_images = len(imlist)

# load vocabulary
with open('vocabulary.pkl', 'rb') as f:
    voc = pickle.load(f)

# create indexer
indx = imagesearch.Indexer('test.db', voc)
indx.create_tables()

# go through all images, project features on vocabulary and insert
for i in range(nbr_images)[:100]:
    locs, descr = sift.read_features_from_file(featlist[i])
    indx.add_to_index(imlist[i], descr)

# commit to database
indx.db_commit()

```

We can now inspect the contents of our database:

```

from pysqlite2 import dbapi2 as sqlite
con = sqlite.connect('test.db')
print con.execute('select count (filename) from imlist').fetchone()
print con.execute('select * from imlist').fetchone()

```

This prints the following to your console:

```

(1000,)
(u'ukbench000000.jpg',)

```

If you try `fetchall()` instead of `fetchone()` in the last line, you will get a long list of all the filenames.

7.4 Searching the Database for Images

With a set of images indexed, we can search the database for similar images. Here we have used a bag-of-word representation for the whole image, but the procedure explained here is generic and can be used to find similar objects, similar faces, similar colors, etc. It all depends on the images and descriptors used.

To handle searches, we introduce a `Searcher` class to *imagesearch.py*:

```
class Searcher(object):

    def __init__(self,db,voc):
        """ Initialize with the name of the database. """
        self.con = sqlite.connect(db)
        self.voc = voc

    def __del__(self):
        self.con.close()
```

A new `Searcher` object connects to the database and closes the connection upon deletion, same as for the `Indexer` class before.

If the number of images is large, it is not feasible to do a full histogram comparison across all images in the database. We need a way to find a reasonably sized set of candidates (where “reasonable” can be determined by search response time, memory requirements, etc.). This is where the word index comes into play. Using the index, we can get a set of candidates and then do the full comparison against that set.

Using the Index to Get Candidates

We can use our index to find all images that contain a particular word. This is just a simple query to the database. Add `candidates_from_word()` as a method for the `Searcher` class:

```
def candidates_from_word(self,imword):
    """ Get list of images containing imword. """

    im_ids = self.con.execute(
        "select distinct imid from imwords where wordid=%d" % imword).fetchall()
    return [i[0] for i in im_ids]
```

This gives the image ids for all images containing the word. To get candidates for more than one word, for example all the nonzero entries in a word histogram, we can loop over each word, get images with that word, and aggregate the lists.³ Here we should

³ If you don't want to use all words, try ranking them according to their idf weight and use the ones with highest weights.

also keep track of how many times each image id appears in the aggregate list, since this shows how many words match the ones in the word histogram. This can be done with the following `Searcher` method:

```
def candidates_from_histogram(self, imwords):
    """ Get list of images with similar words. """

    # get the word ids
    words = imwords.nonzero()[0]

    # find candidates
    candidates = []
    for word in words:
        c = self.candidates_from_word(word)
        candidates+=c

    # take all unique words and reverse sort on occurrence
    tmp = [(w,candidates.count(w)) for w in set(candidates)]
    tmp.sort(cmp=lambda x,y:cmp(x[1],y[1]))
    tmp.reverse()

    # return sorted list, best matches first
    return [w[0] for w in tmp]
```

This method creates a list of word ids from the nonzero entries in a word histogram of an image. Candidates for each word are retrieved and aggregated in the list *candidates*. Then we create a list of tuples (word id, count) with the number of occurrences of each word in the candidate list and sort this list (in place for efficiency) using `sort()` with a custom comparison function that compares the second element in the tuple. The comparison function is declared inline using `lambda` functions, convenient one-line function declarations. The result is returned as a list of image ids with the best matching image first.

Consider the following example:

```
src = imagesearch.Searcher('test.db')
locs,descr = sift.read_features_from_file(featlist[0])
iw = voc.project(descr)

print 'ask using a histogram...'
print src.candidates_from_histogram(iw)[:10]
```

This prints the first 10 lookups from the index and gives the output (this will vary depending on your vocabulary):

```
ask using a histogram...
[655, 656, 654, 44, 9, 653, 42, 43, 41, 12]
```

None of the top 10 candidates are correct. Don't worry; we can now take any number of elements from this list and compare histograms. As you will see, this improves things considerably.

Querying with an Image

There is not much more needed to do a full search using an image as query. To do word histogram comparisons, a `Searcher` object needs to be able to read the image word histograms from the database. Add this method to the `Searcher` class:

```
def get_imhistogram(self, imname):
    """ Return the word histogram for an image. """

    im_id = self.con.execute(
        "select rowid from imlist where filename='%s'" % imname).fetchone()
    s = self.con.execute(
        "select histogram from imhistograms where rowid='%d'" % im_id).fetchone()

    # use pickle to decode NumPy arrays from string
    return pickle.loads(str(s[0]))
```

Again we use `Pickle` to convert between string and `NumPy` arrays, this time with `loads()`.

Now we can combine everything into a query method:

```
def query(self, imname):
    """ Find a list of matching images for imname """

    h = self.get_imhistogram(imname)
    candidates = self.candidates_from_histogram(h)

    matchescores = []
    for imid in candidates:
        # get the name
        cand_name = self.con.execute(
            "select filename from imlist where rowid=%d" % imid).fetchone()
        cand_h = self.get_imhistogram(cand_name)
        cand_dist = sqrt( sum( (h-cand_h)**2 ) ) #use L2 distance
        matchescores.append( (cand_dist, imid) )

    # return a sorted list of distances and database ids
    matchescores.sort()
    return matchescores
```

This `Searcher` method takes the filename of an image and retrieves the word histogram and a list of candidates (which should be limited to some maximum number if you have a large data set). For each candidate, we compare histograms using standard Euclidean distance and return a sorted list of tuples containing distance and image id.

Let's try a query for the same image as in the previous section:

```
src = imagesearch.Searcher('test.db')
print 'try a query...'
print src.query(imlist[0])[0:10]
```

This will again print the top 10 results, including the distance, and should look something like this:

```
try a query...
[(0.0, 1), (100.03999200319841, 2), (105.45141061171255, 3), (129.47200469599596, 708),
(129.73819792181484, 707), (132.68006632497588, 4), (139.89639023220005, 10),
(142.31654858097141, 706), (148.1924424523734, 716), (148.22955170950223, 663)]
```

Much better. The image has distance zero to itself, and two out of the three images of the same scene are on the first two positions. The third image is coming in on position five.

Benchmarking and Plotting the Results

To get a feel for how good the search results are, we can compute the number of correct images on the top four positions. This is the measure used to report performance for the ukbench image set. Here's a function that computes this score. Add it to *imagesearch.py* and you can start optimizing your queries:

```
def compute_ukbench_score(src,imlist):
    """ Returns the average number of correct
        images on the top four results of queries."""

    nbr_images = len(imlist)
    pos = zeros((nbr_images,4))
    # get first four results for each image
    for i in range(nbr_images):
        pos[i] = [w[1]-1 for w in src.query(imlist[i])[:4]]

    # compute score and return average
    score = array([ (pos[i]//4)==(i//4) for i in range(nbr_images)])*1.0
    return sum(score) / (nbr_images)
```

This function gets the top four results and subtracts one from the index returned by *query()* since the database index starts at one and the list of images at zero. Then we compute the score using integer division, using the fact that the correct images are consecutive in groups of four. A perfect result gives a score of 4, nothing right gives a score of 0, and only retrieving the identical images gives a score of 1. Finding the identical image together with two of the three other images gives a score of 3.

Try it out like this:

```
imagesearch.compute_ukbench_score(src,imlist)
```

Or if you don't want to wait (it will take some time to do 1000 queries), just use a subset of the images:

```
imagesearch.compute_ukbench_score(src,imlist[:100])
```

We can consider a score close to 3 as pretty good in this case. The state-of-the-art results reported on the ukbench website are just over 3 (note that they are using more images and your score will decrease with a larger set).

Finally, a function for showing the actual search results will be useful. Add this function,

```
def plot_results(src,res):
    """ Show images in result list 'res'."""

    figure()
    nbr_results = len(res)
    for i in range(nbr_results):
        imname = src.get_filename(res[i])
        subplot(1,nbr_results,i+1)
```

```

        imshow(array(Image.open(imname)))
        axis('off')
    show()

```

which can be called with any number of search results in the list *res*. For example, like this:

```

nbr_results = 6
res = [w[1] for w in src.query(imlist[0][:nbr_results]]
imagesearch.plot_results(src,res)

```

The helper function

```

def get_filename(self,imid):
    """ Return the filename for an image id"""

    s = self.con.execute(
        "select filename from imlist where rowid='%d'" % imid).fetchone()
    return s[0]

```

translates image id to filenames that we need for loading the images when plotting. Some example queries on our data set are shown using `plot_results()` in Figure 7-2.



Figure 7-2. Some example search results on the ukbench data set. The query image is shown on the far left followed by the top five retrieved images.

7.5 Ranking Results Using Geometry

Let's briefly look at a common way of improving results obtained using a bag-of-visual-words model. One of the drawbacks of the model is that the visual words representation of an image does not contain the positions of the image features. This was the price paid to get speed and scalability.

One way to have the feature points improve results is to re-rank the top results using some criteria that takes the features' geometric relationships into account. The most common approach is to fit homographies between the feature locations in the query image and the top result images.

To make this efficient, the feature locations can be stored in the database and correspondences determined by the word id of the features (this only works if the vocabulary is large enough so that the word id matches contain mostly correct matches). This would require a major rewrite of our database and code above and complicate the presentation. To illustrate, we will just reload the features for the top images and match them.

Here is what a complete example of loading all the model files and re-ranking the top results using homographies looks like:

```
import pickle
import sift
import imagesearch
import homography

# load image list and vocabulary
with open('ukbench_imlist.pkl','rb') as f:
    imlist = pickle.load(f)
    featlist = pickle.load(f)

nbr_images = len(imlist)

with open('vocabulary.pkl', 'rb') as f:
    voc = pickle.load(f)

src = imagesearch.Searcher('test.db',voc)

# index of query image and number of results to return
q_ind = 50
nbr_results = 20

# regular query
res_reg = [w[1] for w in src.query(imlist[q_ind]):nbr_results]]
print 'top matches (regular):', res_reg

# load image features for query image
q_locs,q_descr = sift.read_features_from_file(featlist[q_ind])
fp = homography.make_homog(q_locs[:, :2].T)
```



```

# RANSAC model for homography fitting
model = homography.RansacModel()

rank = {}
# load image features for result
for ndx in res_reg[1:]:
    locs,descr = sift.read_features_from_file(featlist[ndx])

    # get matches
    matches = sift.match(q_descr,descr)
    ind = matches.nonzero()[0]
    ind2 = matches[ind]
    tp = homography.make_homog(locs[:, :2].T)

    # compute homography, count inliers. if not enough matches return empty list
    try:
        H,inliers = homography.H_from_ransac(fp[:,ind],tp[:,ind2],model,match_threshold=4)
    except:
        inliers = []

    # store inlier count
    rank[ndx] = len(inliers)

# sort dictionary to get the most inliers first
sorted_rank = sorted(rank.items(), key=lambda t: t[1], reverse=True)
res_geom = [res_reg[0]]+[s[0] for s in sorted_rank]
print 'top matches (homography):', res_geom

# plot the top results
imagesearch.plot_results(src,res_reg[:8])
imagesearch.plot_results(src,res_geom[:8])

```

First, the image list, feature list (containing the filenames of the images and SIFT feature files, respectively), and the vocabulary is loaded. Then a `Searcher` object is created and a regular query is performed and stored in the list `res_reg`. The features for the query image are loaded. Then for each image in the result list, the features are loaded and matched against the query image. Homographies are computed from the matches and the number of inliers counted. If the homography fitting fails, we set the inlier list to an empty list. Finally, we sort the dictionary `rank` that contains image index and inlier count according to decreasing number of inliers. The result lists are printed to the console and the top images visualized.

The output looks like this:

```

top matches (regular): [39, 22, 74, 82, 50, 37, 38, 17, 29, 68, 52, 91, 15, 90, 31, ... ]
top matches (homography): [39, 38, 37, 45, 67, 68, 74, 82, 15, 17, 50, 52, 85, 22, 87, ... ]

```

Figure 7-3 shows some sample results with the regular and re-ranked top images.



Figure 7-3. Some example search results with re-ranking based on geometric consistency using homographies. For each example, the top row is the regular result and the bottom row the re-ranked result.

7.6 Building Demos and Web Applications

In this last section on searching, we'll take a look at a simple way of building demos and web applications with Python. By making demos as web pages, you automatically get cross-platform support and an easy way to show and share your project with minimal requirements. In the sections below we will go through an example of a simple image search engine.

Creating Web Applications with CherryPy

To build these demos, we will use the CherryPy package, available at <http://www.cherrypy.org/>. CherryPy is a pure Python lightweight web server that uses an object oriented model. See Appendix A for more details on how to install and configure CherryPy. Assuming that you have studied the tutorial examples enough to have an initial idea of how CherryPy works, let's build an image search web demo on top of the image searcher you created earlier in this chapter.

Image Search Demo

First, we need to initialize with a few html tags and load the data using Pickle. We need the vocabulary for the Searcher object that interfaces with the database. Create a file `searchdemo.py` and add the following class with two methods:

```

import cherrypy, os, urllib, pickle
import imagesearch

class SearchDemo(object):

    def __init__(self):
        # load list of images
        with open('webimlist.txt') as f:
            self.imlist = f.readlines()

        self.nbr_images = len(self.imlist)
        self.ndx = range(self.nbr_images)

        # load vocabulary
        with open('vocabulary.pkl', 'rb') as f:
            self.voc = pickle.load(f)

        # set max number of results to show
        self.maxres = 15

        # header and footer html
        self.header = """
        <!doctype html>
        <head>
        <title>Image search example</title>
        </head>
        <body>
        """
        self.footer = """
        </body>
        </html>
        """

    def index(self, query=None):
        self.src = imagesearch.Searcher('web.db', self.voc)

        html = self.header
        html += """
        <br />
        Click an image to search. <a href='?query='>Random selection</a> of images.
        <br /><br />
        """

        if query:
            # query the database and get top images
            res = self.src.query(query)[:self.maxres]
            for dist, ndx in res:
                imname = self.src.get_filename(ndx)
                html += "<a href='?query="+imname+"'>"
                html += "<img src='"+imname+"' width='100' />"
                html += "</a>"
        else:
            # show random selection if no query
            random.shuffle(self.ndx)
            for i in self.ndx[:self.maxres]:
                imname = self.imlist[i]
                html += "<a href='?query="+imname+"'>"

```

```

        html += "<img src='"+imname+"' width='100' />"
        html += "</a>"

    html += self.footer
    return html

index.exposed = True

cherry.py.quickstart(SearchDemo(), '/',
    config=os.path.join(os.path.dirname(__file__), 'service.conf'))

```

As you can see, this simple demo consists of a single class with one method for initialization and one for the “index” page (the only page in this case). Methods are automatically mapped to URLs, and arguments to the methods can be passed directly in the URL. The `index` method has a query parameter which, in this case, is the query image to sort the others against. If it is empty, a random selection of images is shown instead. The line

```
index.exposed = True
```

makes the index URL accessible and the last line starts the CherryPy web server with configurations read from *service.conf*. Our configuration file for this example has the following lines:

```

[global]
server.socket_host = "127.0.0.1"
server.socket_port = 8080
server.thread_pool = 50
tools.sessions.on = True

[/]
tools.staticdir.root = "tmp/"
tools.staticdir.on = True
tools.staticdir.dir = ""

```

The first part specifies which IP address and port to use. The second part enables a local folder for reading (in this case “tmp/”). This should be set to the folder containing your images.



Don't put anything secret in that folder if you plan to show this to people. The content of the folder will be accessible through CherryPy.

Start your web server with

```
$ python searchdemo.py
```

from the command line. Opening your browser and pointing it at the right URL (in this case <http://127.0.0.1:8080/>) should show the initial screen with a random selection of images. This should look like the top image in Figure 7-4. Clicking an image starts a query and shows the top results. Clicking an image in the results starts a new query with that image, and so on. There is a link to get back to the starting state of a random selection (by passing an empty query). Some examples are shown in Figure 7-4.

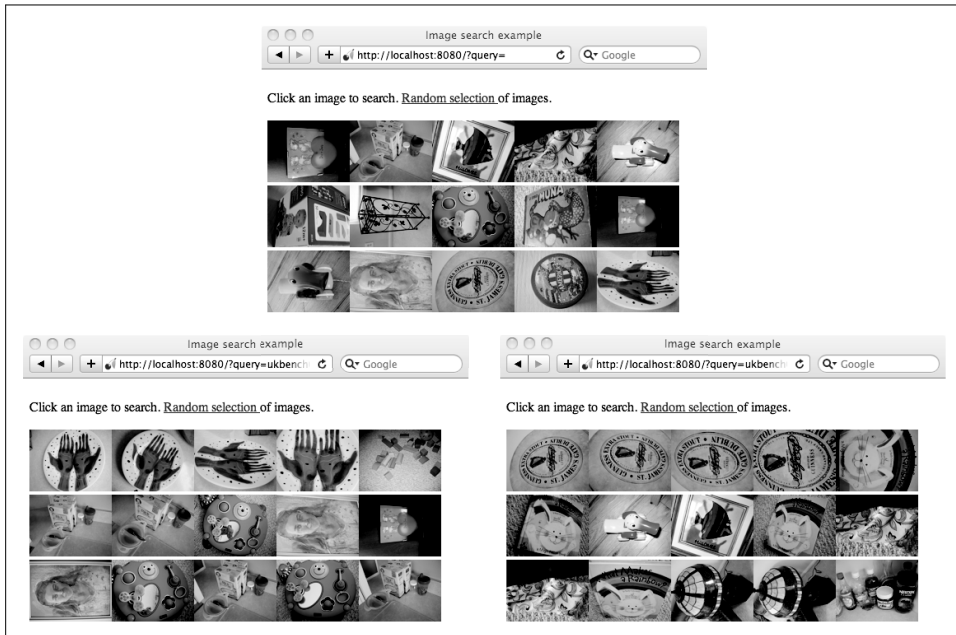


Figure 7-4. Some example search results on the ukbench data set: the starting page, which shows a random selection of the images (top); sample queries (bottom). The query image is shown on the top-left corner followed by the top image results shown row-wise.

This example shows a full integration from web page to database queries and presentation of results. Naturally, we kept the styling and options to a minimum and there are many possibilities for improvement. For example, you may add a stylesheet to make it prettier or upload files to use as queries.

Exercises

1. Try to speed up queries by only using part of the words in the query image to construct the list of candidates. Use the idf weight as a criteria for which words to use.
2. Implement a visual stop word list of the most common visual words in your vocabulary (say, the top 10%) and ignore these words. How does this change the search quality?
3. Visualize a visual word by saving all image features that are mapped to a given word id. Crop image patches around the feature locations (at the given scale) and plot them in a figure. Do the patches for a given word look the same?
4. Experiment with using different distance measures and weighting in the `query()` method. Use the score from `compute_ukbench_score()` to measure your progress.

5. Throughout this chapter, we only used SIFT features in our vocabulary. This completely disregards the color information, as you can see in the example results in Figure 7-2. Try to add color descriptors and see if you can improve the search results.
6. For large vocabularies, using arrays to represent the visual word frequencies is inefficient, since most of the entries will be zero (think of the case with a few hundred thousand words and images with typically a thousand features). One way to overcome this inefficiency is to use dictionaries as sparse array representations. Replace the arrays with a sparse class of your own and add the necessary methods. Alternatively, try to use the `scipy.sparse` module.
7. As you try to increase the size of the vocabulary, the clustering time will take too long and the projection of features to words also becomes slower. Implement a hierarchical vocabulary using hierarchical k -mean clustering and see how this improves scaling. See the paper [23] for details and inspiration.

Classifying Image Content

This chapter introduces algorithms for classifying images and image content. We look at some simple but effective methods as well as state-of-the-art classifiers and apply them to two-class and multi-class problems. We show examples with applications in gesture recognition and object recognition.

8.1 *K*-Nearest Neighbors

One of the simplest and most used methods for classification is the *k-nearest neighbor classifier* (*kNN*). The algorithm simply compares an object (for example a feature vector) to be classified with all objects in a training set with known class labels and lets the *k* nearest vote for which class to assign. This method often performs well but has a number of drawbacks. Same as with the *k*-means clustering algorithm, the number *k* needs to be chosen and the value will affect performance. Furthermore, the method requires the entire training set to be stored, and if this set is large, it will be slow to search. For large training sets some form of binning is usually used to reduce the number of comparisons needed.¹ On the positive side, there are no restrictions on what distance measure to use; practically anything you can think of will work (which is not the same as saying that it will perform well). The algorithm is also trivially parallelizable.

Implementing *kNN* in a basic form is pretty straightforward. Given a set of training examples and a list of associated labels, the code below does the job. The training examples and labels can be rows in an array or just in a list. They can be numbers, strings, whatever you like. Add this class to a file called *knn.py*:

```
class KnnClassifier(object):

    def __init__(self, labels, samples):
        """ Initialize classifier with training data. """

        self.labels = labels
        self.samples = samples
```

¹ Another option is to only keep a selected subset of the training set. This can, however, impact accuracy.

```

def classify(self,point,k=3):
    """ Classify a point against k nearest
        in the training data, return label. """

    # compute distance to all training points
    dist = array([L2dist(point,s) for s in self.samples])

    # sort them
    ndx = dist.argsort()

    # use dictionary to store the k nearest
    votes = {}
    for i in range(k):
        label = self.labels[ndx[i]]
        votes.setdefault(label,0)
        votes[label] += 1

    return max(votes)

def L2dist(p1,p2):
    return sqrt( sum( (p1-p2)**2) )

```

It is easiest to define a class and initialize with the training data. That way, we don't have to store and pass the training data as arguments every time we want to classify something. Using a dictionary for storing the k nearest labels makes it possible to have labels as text strings or numbers. In this example, we used the Euclidean (L_2) distance measure. If you have other measures, just add them as functions.

A Simple 2D Example

Let's first create some simple 2D example data sets to illustrate and visualize how this classifier works. The following script will create two different 2D point sets, each with two classes, and save the data using Pickle:

```

from numpy.random import randn
import pickle

# create sample data of 2D points
n = 200

# two normal distributions
class_1 = 0.6 * randn(n,2)
class_2 = 1.2 * randn(n,2) + array([5,1])
labels = hstack((ones(n),-ones(n)))

# save with Pickle
with open('points_normal.pkl', 'w') as f:
    pickle.dump(class_1,f)
    pickle.dump(class_2,f)
    pickle.dump(labels,f)

# normal distribution and ring around it
class_1 = 0.6 * randn(n,2)

```



```

r = 0.8 * randn(n,1) + 5
angle = 2*pi * randn(n,1)
class_2 = hstack((r*cos(angle),r*sin(angle)))
labels = hstack((ones(n),-ones(n)))

# save with Pickle
with open('points_ring.pkl', 'w') as f:
    pickle.dump(class_1,f)
    pickle.dump(class_2,f)
    pickle.dump(labels,f)

```

Run the script twice with different filenames, for example *points_normal_test.pkl* first and *points_ring_test.pkl* the second time. You will now have four files with 2D data sets, two files for each of the distributions. We can use one for training and the other for testing.

Let's see how to do that with the kNN classifier. Create a script with the following commands:

```

import pickle
import knn
import imtools

# load 2D points using Pickle
with open('points_normal.pkl', 'r') as f:
    class_1 = pickle.load(f)
    class_2 = pickle.load(f)
    labels = pickle.load(f)

model = knn.KnnClassifier(labels,vstack((class_1,class_2)))

```

This will create a kNN classifier *model* using the data in the Pickle file. Now add the following:

```

# load test data using Pickle
with open('points_normal_test.pkl', 'r') as f:
    class_1 = pickle.load(f)
    class_2 = pickle.load(f)
    labels = pickle.load(f)

# test on the first point
print model.classify(class_1[0])

```

This loads the other data set (the test set) and prints the estimated class label of the first point to your console.

To visualize the classification of all the test points and show how well the classifier separates the two classes, we can add these lines:

```

# define function for plotting
def classify(x,y,model=model):
    return array([model.classify([xx,yy]) for (xx,yy) in zip(x,y)])

# plot the classification boundary
imtools.plot_2D_boundary([-6,6,-6,6],[class_1,class_2],classify,[1,-1])
show()

```

Here we created a small helper function that takes arrays of 2D coordinates *x* and *y* and the classifier and returns an array of estimated class labels. Now we can pass this function as an argument to the actual plotting function. Add the following function to your file *imtools*:

```
def plot_2D_boundary(plot_range,points,decisionfcn,labels,values=[0]):
    """ Plot range is (xmin,xmax,ymin,ymax), points is a list
        of class points, decisionfcn is a funtion to evaluate,
        labels is a list of labels that decisionfcn returns for each class,
        values is a list of decision contours to show. """

    clist = ['b','r','g','k','m','y'] # colors for the classes

    # evaluate on a grid and plot contour of decision function
    x = arange(plot_range[0],plot_range[1],.1)
    y = arange(plot_range[2],plot_range[3],.1)
    xx,yy = meshgrid(x,y)
    xxx,yyy = xx.flatten(),yy.flatten() # lists of x,y in grid
    zz = array(decisionfcn(xxx,yyy))
    zz = zz.reshape(xx.shape)
    # plot contour(s) at values
    contour(xx,yy,zz,values)

    # for each class, plot the points with '*' for correct, 'o' for incorrect
    for i in range(len(points)):
        d = decisionfcn(points[i][:,0],points[i][:,1])
        correct_ndx = labels[i]==d
        incorrect_ndx = labels[i]!=d
        plot(points[i][correct_ndx,0],points[i][correct_ndx,1], '*',color=clist[i])
        plot(points[i][incorrect_ndx,0],points[i][incorrect_ndx,1], 'o',color=clist[i])

    axis('equal')
```

This function takes a decision function (the classifier) and evaluates it on a grid using `meshgrid()`. The contours of the decision function can be plotted to show where the boundaries are. The default is the zero contour. The resulting plots look like the ones in Figure 8-1. As you can see, the kNN decision boundary can adapt to the distribution of the classes without any explicit modeling.

Dense SIFT as Image Feature

Let's look at classifying some images. To do so, we need a feature vector for the image. We saw feature vectors with average RGB and PCA coefficients as examples in the clustering chapter. Here we will introduce another representation, the *dense SIFT* feature vector.

A dense SIFT representation is created by applying the descriptor part of SIFT to a regular grid across the whole image.² We can use the same executables as in Section 2.2 and get dense SIFT features by adding some extra parameters. Create a file *dsift.py* as a placeholder for the dense SIFT computation and add the following function:

² Another common name is *Histogram of Oriented Gradients* (HOG).

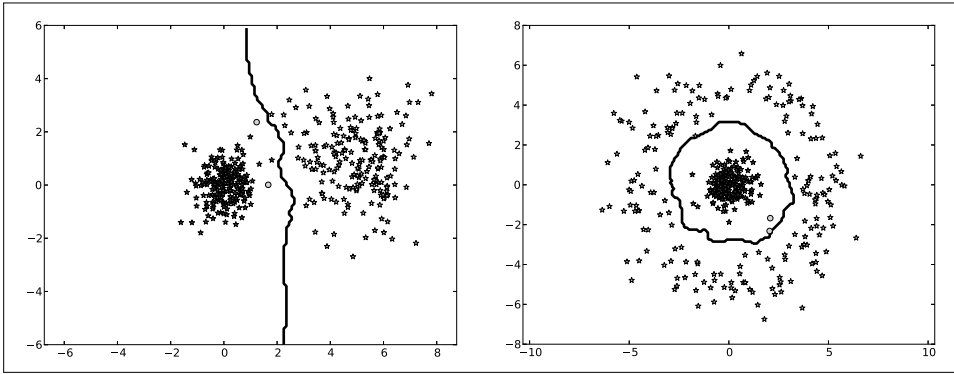


Figure 8-1. Classifying 2D data using a k -nearest neighbor classifier. For each example, the color shows the class label. Correctly classified points are shown with stars and misclassified points with circles. The curve is the classifier decision boundary.

```
import sift

def process_image_dsift(imagename, resultname, size=20, steps=10,
                        force_orientation=False, resize=None):
    """ Process an image with densely sampled SIFT descriptors
    and save the results in a file. Optional input: size of features,
    steps between locations, forcing computation of descriptor orientation
    (False means all are oriented upward), tuple for resizing the image."""

    im = Image.open(imagename).convert('L')
    if resize != None:
        im = im.resize(resize)
    m, n = im.size

    if imagename[-3:] != '.pgm':
        # create a pgm file
        im.save('tmp.pgm')
        imagename = 'tmp.pgm'

    # create frames and save to temporary file
    scale = size/3.0
    x, y = meshgrid(range(steps, m, steps), range(steps, n, steps))
    xx, yy = x.flatten(), y.flatten()
    frame = array([xx, yy, scale*ones(xx.shape[0]), zeros(xx.shape[0])])
    savetxt('tmp.frame', frame.T, fmt='%03.3f')

    if force_orientation:
        cmd = "sift " + imagename + " --output=" + resultname +
            " --read-frames=tmp.frame --orientations"
    else:
        cmd = "sift " + imagename + " --output=" + resultname +
            " --read-frames=tmp.frame"
    os.system(cmd)
    print 'processed', imagename, 'to', resultname
```

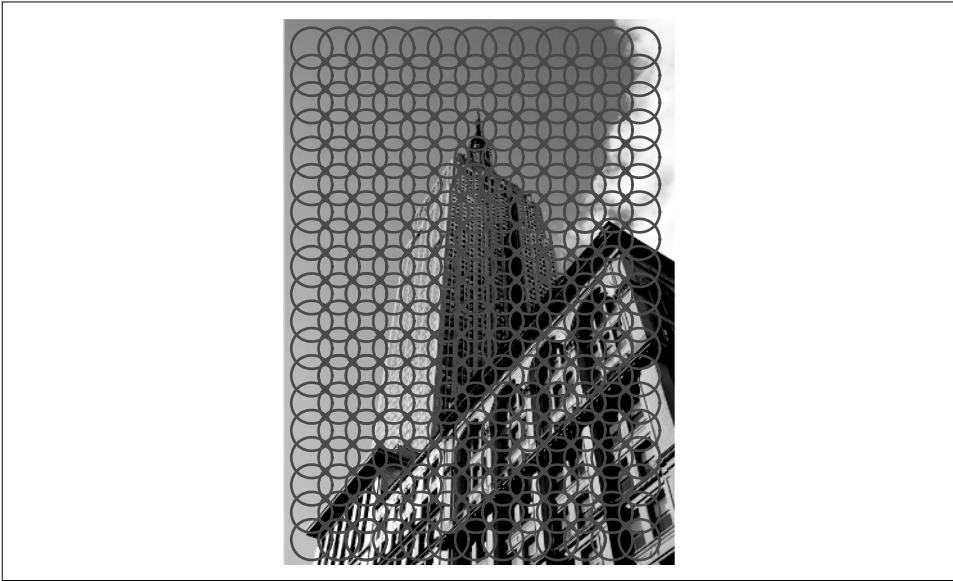


Figure 8-2. An example of applying dense SIFT descriptors over an image.

Compare this to the function `process_image()` in Section 2.2. We use the function `save_txt()` to store the *frame* array in a text file for command line processing. The last parameter of this function can be used to resize the image before extracting the descriptors. For example, passing `imsize=(100, 100)` will resize to square images 100×100 pixels. Last, if `force_orientation` is true the descriptors will be normalized based on the local dominant gradient direction. If it is false, all descriptors are simply oriented upward.

Use it like this to compute the dense SIFT descriptors and visualize the locations:

```
import dsift,sift

dsift.process_image_dsift('empire.jpg','empire.sift',90,40,True)
l,d = sift.read_features_from_file('empire.sift')

im = array(Image.open('empire.jpg'))
sift.plot_features(im,l,True)
show()
```

This will compute SIFT features densely across the image with the local gradient orientation used to orient the descriptors (by setting `force_orientation` to true). The locations are shown in Figure 8-2.

Classifying Images—Hand Gesture Recognition

In this application, we will look at applying the dense SIFT descriptor to images of hand gestures to build a simple hand gesture recognition system. We will use some images from the Static Hand Posture Database (available at <http://www.idiap.ch/resource/gestures/>) to illustrate. Download the smaller test set (“test set 16.3Mb” on the web page)

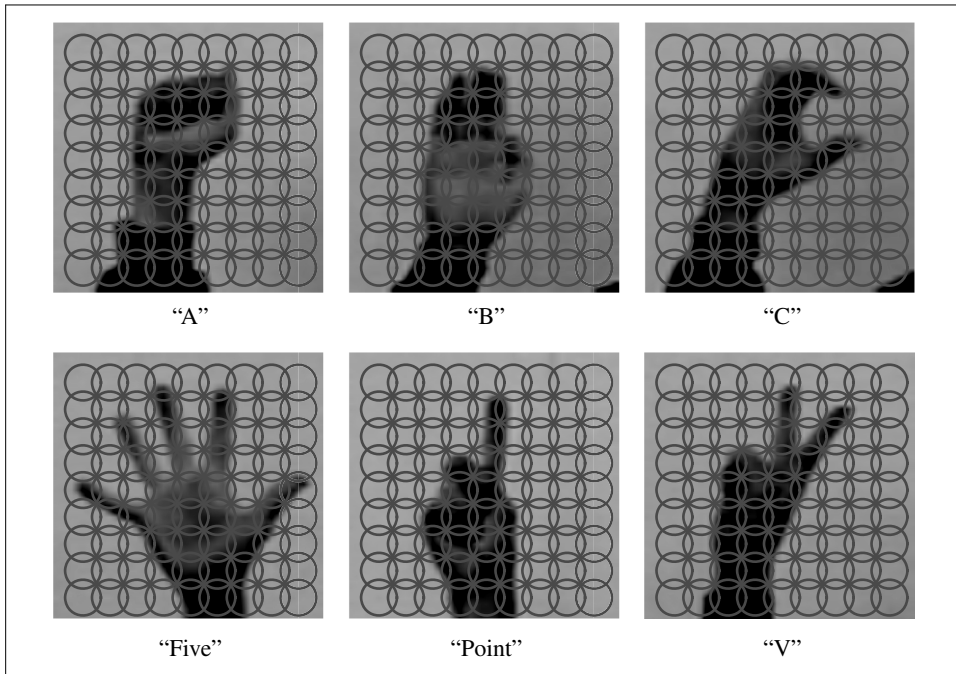


Figure 8-3. Dense SIFT descriptors on sample images from the six categories of hand gesture images. (images from the Static Hand Posture Database)

and take all the images in the “uniform” folders and split each class evenly into two folders called “train” and “test”.

Process the images with the dense SIFT function above to get feature vectors for all images. Again, assuming the filenames are in a list *imlist*, this is done like this:

```
import dsift

# process images at fixed size (50,50)
for filename in imlist:
    featfile = filename[:-3]+'dsift'
    dsift.process_image_dsift(filename,featfile,10,5,resize=(50,50))
```

This creates feature files for each image with the extension “.dsift”. *Note the resizing of the images to some common fixed size.* This is very important; otherwise your images will have varying number of descriptors, and therefore varying length of the feature vectors. This will cause errors when comparing them later. Plotting the images with the descriptors looks like Figure 8-3.

Define a helper function for reading the dense SIFT descriptor files like this:

```
import os, sift

def read_gesture_features_labels(path):
    # create list of all files ending in .dsift
    featlist = [os.path.join(path,f) for f in os.listdir(path) if f.endswith('.dsift')]
```

```

# read the features
features = []
for featfile in featlist:
    l,d = sift.read_features_from_file(featfile)
    features.append(d.flatten())
features = array(features)

# create labels
labels = [featfile.split('/')[-1][0] for featfile in featlist]

return features,array(labels)

```

Then we can read the features and labels for our test and training sets using the following commands:

```

features,labels = read_gesture_features_labels('train/')

test_features,test_labels = read_gesture_features_labels('test/')

classnames = unique(labels)

```

Here we used the first letter in the filename to create class labels. Using the NumPy function `unique()`, we get a sorted list of unique class names.

Now we can try our nearest neighbor code on this data:

```

# test kNN
k = 1
knn_classifier = knn.KnnClassifier(labels,features)
res = array([knn_classifier.classify(test_features[i],k) for i in
             range(len(test_labels))])

# accuracy
acc = sum(1.0*(res==test_labels)) / len(test_labels)
print 'Accuracy:', acc

```

First, the classifier object is created with the training data and labels as input. Then we iterate over the test set and classify each image using the `classify()` method. The accuracy is computed by multiplying the boolean array by one and summing. In this case, the true values are 1, so it is a simple thing to count the correct classifications. This gives a printout like:

```
Accuracy: 0.811518324607
```

which means that 81% were correctly classified in this case. The value will vary with the choice of k and the parameters for the dense SIFT image descriptor.

The accuracy above shows how many correct classifications there are for a given test set, but it does not tell us which signs are hard to classify or which mistakes are typically made. A *confusion matrix* is a matrix that shows how many samples of each class are classified as each of the classes. It shows how the errors are distributed and what classes are often “confused” for each other.

The following function will print the labels and the corresponding confusion matrix:

```
def print_confusion(res, labels, classnames):

    n = len(classnames)

    # confusion matrix
    class_ind = dict([(classnames[i], i) for i in range(n)])

    confuse = zeros((n, n))
    for i in range(len(test_labels)):
        confuse[class_ind[res[i]], class_ind[test_labels[i]]] += 1

    print 'Confusion matrix for'
    print classnames
    print confuse
```

The printout of running

```
print_confusion(res, test_labels, classnames)
```

looks like this:

```
Confusion matrix for
['A' 'B' 'C' 'F' 'P' 'V']
[[ 26.  0.  2.  0.  1.  1.]
 [  0. 26.  0.  1.  1.  1.]
 [  0.  0. 25.  0.  0.  1.]
 [  0.  3.  0. 37.  0.  0.]
 [  0.  1.  2.  0. 17.  1.]
 [  3.  1.  3.  0. 14. 24.]]
```

This shows that, for example, in this case “P” (“Point”) is often misclassified as “V.”

8.2 Bayes Classifier

Another simple but powerful classifier is the *Bayes classifier*³ (or *naive Bayes classifier*). A Bayes classifier is a probabilistic classifier based on applying Bayes’ theorem for conditional probabilities. The assumption is that all features are independent and unrelated to each other (this is the “naive” part). Bayes classifiers can be trained very efficiently, since the model chosen is applied to each feature independently. Despite their simplistic assumptions, Bayes classifiers have been very successful in practical applications, in particular for email spam filtering. Another benefit of this classifier is that once the model is learned, no training data needs to be stored. Only the model parameters are needed.

The classifier is constructed by multiplying the individual conditional probabilities from each feature to get the total probability of a class. Then the class with highest probability is selected.

³ After Thomas Bayes, an 18th-century English mathematician and minister.

Let's look at a basic implementation of a Bayes classifier using Gaussian probability distribution models. This means that each feature is individually modeled using the feature mean and variance, computed from a set of training data. Add the following classifier class to a file called *bayes.py*:

```
class BayesClassifier(object):

    def __init__(self):
        """ Initialize classifier with training data. """

        self.labels = [] # class labels
        self.mean = [] # class mean
        self.var = [] # class variances
        self.n = 0 # nbr of classes

    def train(self,data,labels=None):
        """ Train on data (list of arrays n*dim).
            Labels are optional, default is 0...n-1. """

        if labels==None:
            labels = range(len(data))
            self.labels = labels
            self.n = len(labels)

        for c in data:
            self.mean.append(mean(c,axis=0))
            self.var.append(var(c,axis=0))

    def classify(self,points):
        """ Classify the points by computing probabilities
            for each class and return most probable label. """

        # compute probabilities for each class
        est_prob = array([gauss(m,v,points) for m,v in zip(self.mean,self.var)])

        # get index of highest probability, this gives class label
        ndx = est_prob.argmax(axis=0)
        est_labels = array([self.labels[n] for n in ndx])

        return est_labels, est_prob
```

The model has two variables per class, the class mean and covariance. The `train()` method takes a lists of feature arrays (one per class) and computes the mean and covariance for each. The method `classify()` computes the class probabilities for an array of data points and selects the class with highest probability. The estimated class labels and probabilities are returned. The helper function for the actual Gaussian function is also needed:

```
def gauss(m,v,x):
    """ Evaluate Gaussian in d-dimensions with independent
        mean m and variance v at the points in (the rows of) x. """

    if len(x.shape)==1:
        n,d = 1,x.shape[0]
    else:
        n,d = x.shape

    n,d = x.shape
```



```

# covariance matrix, subtract mean
S = diag(1/v)
x = x-m
# product of probabilities
y = exp(-0.5*diag(dot(x,dot(S,x.T))))

# normalize and return
return y * (2*pi)**(-d/2.0) / ( sqrt(prod(v)) + 1e-6)

```

This function computes the product of the individual Gaussian distributions and returns the probability for a given pair of model parameters m, v . For more details on this function, see for example http://en.wikipedia.org/wiki/Multivariate_normal_distribution.

Try this Bayes classifier on the 2D data from the previous section. This script will load the exact same point sets and train a classifier:

```

import pickle
import bayes
import imtools

# load 2D example points using Pickle
with open('points_normal.pkl', 'r') as f:
    class_1 = pickle.load(f)
    class_2 = pickle.load(f)
    labels = pickle.load(f)

# train Bayes classifier
bc = bayes.BayesClassifier()
bc.train([class_1,class_2],[1,-1])

```

Now, we can load the other one and test the classifier:

```

# load test data using Pickle
with open('points_normal_test.pkl', 'r') as f:
    class_1 = pickle.load(f)
    class_2 = pickle.load(f)
    labels = pickle.load(f)

# test on some points
print bc.classify(class_1[:10])[0]

# plot points and decision boundary
def classify(x,y,bc=bc):
    points = vstack((x,y))
    return bc.classify(points.T)[0]

imtools.plot_2D_boundary([-6,6,-6,6],[class_1,class_2],classify,[1,-1])
show()

```

This prints the classification result for the first 10 points to the console. It might look like this:

```
[1 1 1 1 1 1 1 1 1 1]
```

Again, we used a helper function `classify()` to pass to the plotting function for visualizing the classification results by evaluating the function on a grid. The plots for the

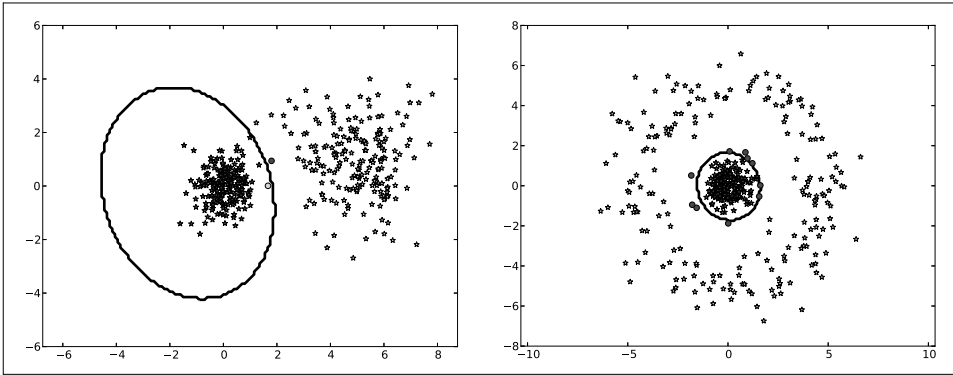


Figure 8-4. Classifying 2D data using a Bayes classifier. For each example, the color shows the class label. Correctly classified points are shown with stars and misclassified points with circles. The curve is the classifier decision boundary.

two sets look like Figure 8-4. The decision boundary, in this case, will be the ellipse-like level curves of a 2D Gaussian function.

Using PCA to Reduce Dimensions

Now, let's try the gesture recognition problem. Since the feature vectors are very large for the dense SIFT descriptor (more than 10,000 for the parameter choices in the example above), it is a good idea to do dimensionality reduction before fitting models to the data. Principal Component Analysis, PCA, (see Section 1.3) usually does a good job. Try the following script that uses PCA from the file *pca.py* (page 13):

```
import pca

V,S,m = pca.pca(features)

# keep most important dimensions
V = V[:50]
features = array([dot(V,f-m) for f in features])
test_features = array([dot(V,f-m) for f in test_features])
```

Here *features* and *test_features* are the same arrays that we loaded for the kNN example. In this case, we apply PCA on the training data and keep the 50 dimensions with most variance. This is done by subtracting the mean *m* (computed on the training data) and multiplying with the basis vectors *V*. The same transformation is applied to the test data.

Train and test the Bayes classifier like this:

```
# test Bayes
bc = bayes.BayesClassifier()
blist = [features[where(labels==c)[0]] for c in classnames]

bc.train(blist,classnames)
res = bc.classify(test_features)[0]
```

Since `BayesClassifier` takes a list of arrays (one array for each class), we transform the data before passing it to the `train()` method. Since we don't need the probabilities for now, we chose to return only the labels of the classification.

Checking the accuracy

```
acc = sum(1.0*(res==test_labels)) / len(test_labels)
print 'Accuracy:', acc
```

gives something like this:

```
Accuracy: 0.717277486911
```

Checking the confusion matrix

```
print_confusion(res, test_labels, classnames)
```

gives a print out like this:

```
Confusion matrix for
['A' 'B' 'C' 'F' 'P' 'V']
[[ 20.  0.  0.  4.  0.  0.]
 [ 0. 26.  1.  7.  2.  2.]
 [ 1.  0. 27.  5.  1.  0.]
 [ 0.  2.  0. 17.  0.  0.]
 [ 0.  1.  0.  4. 22.  1.]
 [ 8.  2.  4.  1.  8. 25.]]
```

This is not as good as the kNN classifier, but with the Bayes classifier we don't need to keep any training data, just the model parameters for each of the classes. The result will vary greatly with the choice of dimensions after PCA.

8.3 Support Vector Machines

Support Vector Machines (SVM) are a powerful type of classifiers that often give state-of-the-art results for many classification problems. In its simplest form, an SVM finds a linear separating hyperplane (a plane in higher-dimensional spaces) with the best possible separation between two classes. The decision function for a feature vector $\mathbf{x}$ is

$$f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} - b,$$

where $\mathbf{w}$ is the hyperplane normal and b an offset constant. The zero level of this function then ideally separates the two classes so that one class has positive values and the other negative. The parameters $\mathbf{w}$ and b are found by solving an optimization problem on a training set of labeled feature vectors $\mathbf{x}_i$ with labels $y_i \in \{-1, 1\}$ so that the hyperplane has maximal separation between the two classes. The normal is a linear combination of some of the training feature vectors

$$\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i,$$

so that the decision function can be written

$$f(\mathbf{x}) = \sum_i \alpha_i y_i \mathbf{x}_i \cdot \mathbf{x} - b.$$

Here i runs over a selection of the training vectors; the selected vectors $\mathbf{x}_i$ are called *support vectors* since they help define the classification boundary.

One of the strengths of SVM is that by using *kernel functions*, that is, functions that map the feature vectors to a different (higher) dimensional space, non-linear or very difficult classification problems can be effectively solved while still keeping some control over the decision function. Kernel functions replace the inner product of the classification function, $\mathbf{x}_i \cdot \mathbf{x}$, with a function $K(\mathbf{x}_i, \mathbf{x})$.

Some of the most common kernel functions are:

- *linear*, a hyperplane in feature space, the simplest case, $K(\mathbf{x}_i, \mathbf{x}) = \mathbf{x}_i \cdot \mathbf{x}$
- *polynomial*, features are mapped with polynomials of a defined degree d , $K(\mathbf{x}_i, \mathbf{x}) = (\gamma \mathbf{x}_i \cdot \mathbf{x} + r)^d$, $\gamma > 0$
- *radial basis functions*, exponential functions, usually a very effective choice, $K(\mathbf{x}_i, \mathbf{x}) = e^{(-\gamma \|\mathbf{x}_i - \mathbf{x}\|^2)}$, $\gamma > 0$
- *sigmoid*, a smoother alternative to hyperplane, $K(\mathbf{x}_i, \mathbf{x}) = \tanh(\gamma \mathbf{x}_i \cdot \mathbf{x} + r)$

The parameters of each kernel are also determined during training.

For multi-class problems, the usual procedure is to train multiple SVMs so that each separates one class from the rest (also known as “one-versus-all” classifiers). For more details on SVMs, see for example the book [9] and the online references at <http://www.support-vector.net/references.html>.

Using LibSVM

We will use one of the best and most commonly used implementations available, LibSVM [7]. LibSVM comes with a nice Python interface (there are also interfaces for many other programming languages). For installation instructions, see Section A.4.

Let’s use LibSVM on the sample 2D point data to see how it works. This script will load the same points and train an SVM classifier using radial basis functions:

```
import pickle
from svmutil import *
import itertools

# load 2D example points using Pickle
with open('points_normal.pkl', 'r') as f:
    class_1 = pickle.load(f)
    class_2 = pickle.load(f)
    labels = pickle.load(f)

# convert to lists for libsvm
class_1 = map(list, class_1)
class_2 = map(list, class_2)
labels = list(labels)
samples = class_1 + class_2 # concatenate the two lists

# create SVM
prob = svm_problem(labels, samples)
param = svm_parameter('-t 2')
```

```
# train SVM on data
m = svm_train(prob,param)

# how did the training do?
res = svm_predict(labels,samples,m)
```

Loading the data set is the same as before, but this time we have to convert the arrays to lists since LibSVM does not support array objects as input. Here we used Python's built-in `map()` function that applies the conversion function `list()` to each element. The next lines create a SVM problem object and sets some parameters. The `svm_train()` call solves the optimization problem for determining the model parameters. The model can then be used for predictions. The last call to `svm_predict()` will classify the training data with the model *m* and shows how successful the training was. The printout looks something like this:

```
Accuracy = 100% (400/400) (classification)
```

This means that the classifier completely separates the training data and correctly classifies all 400 data points.

Note that we added a string of parameter choices in the call to train the classifier. These parameters are used to control the kernel type, degree, and other choices for the classifier. Most of them are outside the scope of this book but the important ones to know are “t” and “k”. The parameter “t” determines the type of kernel used. The options are:

“t”	kernel
0	linear
1	polynomial
2	radial basis function (default)
3	sigmoid

The parameter “k” determines the degree of the polynomial (default is 3).

Now, load the other point set and test the classifier:

```
# load test data using Pickle
with open('points_normal_test.pkl', 'r') as f:
    class_1 = pickle.load(f)
    class_2 = pickle.load(f)
    labels = pickle.load(f)

# convert to lists for libsvm
class_1 = map(list,class_1)
class_2 = map(list,class_2)

# define function for plotting
def predict(x,y,model=m):
    return array(svm_predict([0]*len(x),zip(x,y),model)[0])

# plot the classification boundary
imtools.plot_2D_boundary([-6,6,-6,6],[array(class_1),array(class_2)],predict,[-1,1])
show()
```

Again we have to convert the data to lists for LibSVM. As before, we also define a helper function `predict()` for plotting the classification boundary. Note the use of a list of zeros

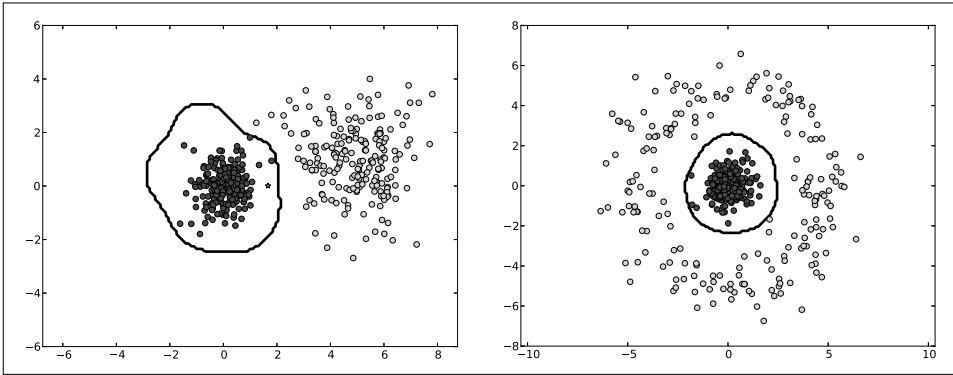


Figure 8-5. Classifying 2D data using a Support Vector Machine classifier. For each example, the color shows the class label. Correctly classified points are shown with stars and misclassified points with circles. The curve is the classifier decision boundary.

`[0]*len(x)` as a replacement for the label list if true labels are unavailable. You can use any list as long as it has the correct length. The 2D plots for the two different point data sets are shown in Figure 8-5.

Hand Gesture Recognition Again

Using LibSVM on our multi-class hand gesture recognition problem is fairly straightforward. Multiple classes are automatically handled, so we only need to format the data so that the input and output matches the requirements of LibSVM.

With training and testing data in arrays named *features* and *test_features* as in the previous examples, the following will load the data and train a linear SVM classifier:

```
features = map(list,features)
test_features = map(list,test_features)

# create conversion function for the labels
transl = {}
for i,c in enumerate(classnames):
    transl[c],transl[i] = i,c

# create SVM
prob = svm_problem(convert_labels(labels,transl),features)
param = svm_parameter('-t 0')

# train SVM on data
m = svm_train(prob,param)

# how did the training do?
res = svm_predict(convert_labels(labels,transl),features,m)

# test the SVM
res = svm_predict(convert_labels(test_labels,transl),test_features,m)[0]
res = convert_labels(res,transl)
```

Same as before, we convert the data to lists using a `map()` call. Then the labels need to be converted since LibSVM does not handle string labels. The dictionary *transl* will contain a conversion between string and integer labels. Try to print it to your console to see what happens. The parameter “-t 0” makes it a linear classifier and the decision boundary will be a hyperplane in the original feature space of some 10,000 dimensions.

Now compare the labels, just like before:

```
acc = sum(1.0*(res==test_labels)) / len(test_labels)
print 'Accuracy:', acc

print_confusion(res,test_labels,classnames)
```

The output using this linear kernel should look like this:

```
Accuracy: 0.916230366492
Confusion matrix for
['A' 'B' 'C' 'F' 'P' 'V']
[[ 26.  0.  1.  0.  2.  0.]
 [  0. 28.  0.  0.  1.  0.]
 [  0.  0. 29.  0.  0.  0.]
 [  0.  2.  0. 38.  0.  0.]
 [  0.  1.  0.  0. 27.  1.]
 [  3.  0.  2.  0.  3. 27.]]
```

Now if we apply PCA to reduce the dimensions to 50, as we did in Section 8.2, this changes the accuracy to:

```
Accuracy: 0.890052356021
```

Not bad, seeing that the feature vectors are about 200 times smaller than the original data (and the space to store the support vectors then also 200 times less).

8.4 Optical Character Recognition

As an example of a multi-class problem, let's look at interpreting images of Sudokus. *Optical character recognition* (OCR) is the process of interpreting images of hand- or machine-written text. A common example is text extraction from scanned documents such as zip-codes on letters or book pages such as the library volumes in Google Books (<http://books.google.com/>). Here we will look at a simple OCR problem of recognizing numbers in images of printed Sudokus. Sudokus are a form of logic puzzles where the goal is to fill a 9×9 grid with the numbers $1 \dots 9$ so that each column, each row, and each 3×3 sub-grid contains all nine digits.⁴ In this example, we are just interested in reading the puzzle and interpreting it correctly. Actually solving the puzzle, we leave to you.

Training a Classifier

For this classification problem we have ten classes, the numbers $1 \dots 9$, and the empty cells. Let's give the empty cells the label 0 so that our class labels are $0 \dots 9$. To train

⁴ See <http://en.wikipedia.org/wiki/Sudoku> for more details if you are unfamiliar with the concept.

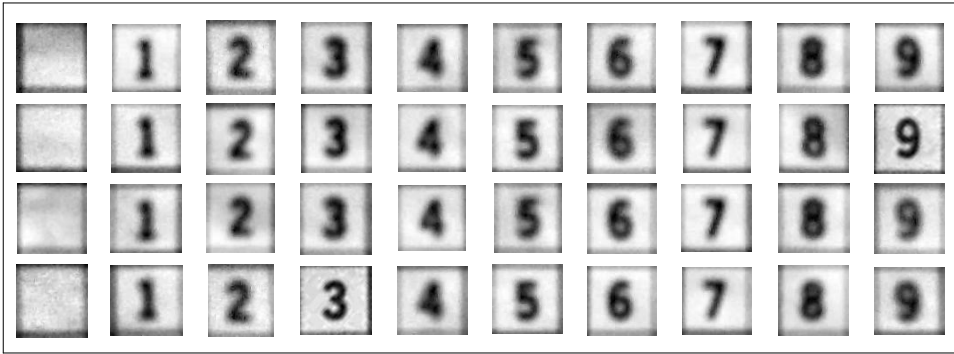


Figure 8-6. Sample training images for the 10 classes of the Sudoku OCR classifier.

this ten-class classifier, we will use a dataset of images of cropped Sudoku cells.⁵ In the file *sudoku_images.zip* are two folders, “ocr_data” and “sudokus”. The latter contains images of Sudokus under varying conditions. We will save those for later. For now, take a look at the folder “ocr_data”. It contains two subfolders with images, one for training and one for testing. The images are named with the first character equal to the class (0 . . . 9). Figure 8-6 shows some samples from the training set. The images are grayscale and roughly 80×80 pixels (with some variation).

Selecting Features

We need to decide on what feature vector to use for representing each cell image. There are many good choices; here we’ll try something simple but still effective. The following function takes an image and returns a feature vector of the flattened grayscale values:

```
def compute_feature(im):
    """ Returns a feature vector for an
        ocr image patch. """

    # resize and remove border
    norm_im = imresize(im,(30,30))
    norm_im = norm_im[3:-3,3:-3]

    return norm_im.flatten()
```

This function uses the resizing function *imresize()* from *imtools* to reduce the length of the feature vector. We also crop away about 10% border pixels since the crops often get parts of the grid lines on the edges, as you can see in Figure 8-6.

Now we can read the training data using a function like this:

```
def load_ocr_data(path):
    """ Return labels and ocr features for all images
        in path. """
```

⁵ Images courtesy of Martin Byröd [4], <http://www.maths.lth.se/matematiklth/personal/byrod/>, collected and cropped from photos of actual Sudokus.


```

# create list of all files ending in .jpg
imlist = [os.path.join(path,f) for f in os.listdir(path) if f.endswith('.jpg')]
# create labels
labels = [int(imfile.split('/')[0]) for imfile in imlist]

# create features from the images
features = []
for imname in imlist:
    im = array(Image.open(imname).convert('L'))
    features.append(compute_feature(im))
return array(features),labels

```

The labels are extracted as the first character of the filename of each of the JPEG files and stored in the *labels* list as integers. The feature vectors are computed using the function above and stored in an array.

Multi-Class SVM

With the training data in place, we are ready to learn a classifier. Here we'll use a multi-class support vector machine. The code looks just as it does in the previous section:

```

from svmutil import *

# TRAINING DATA
features,labels = load_ocr_data('training/')

# TESTING DATA
test_features,test_labels = load_ocr_data('testing/')

# train a linear SVM classifier
features = map(list,features)
test_features = map(list,test_features)

prob = svm_problem(labels,features)
param = svm_parameter('-t 0')

m = svm_train(prob,param)

# how did the training do?
res = svm_predict(labels,features,m)

# how does it perform on the test set?
res = svm_predict(test_labels,test_features,m)

```

This trains a linear SVM classifier and tests the performance on the unseen images in the test set. You should get the following printout from the last two `svm_predict()` calls:

```

Accuracy = 100% (1409/1409) (classification)
Accuracy = 99.2979% (990/997) (classification)

```

Great news. The 1,409 images of the training set are perfectly separated in the ten classes and the recognition performance on the test set is around 99%. We can now use this classifier on crops from new Sudoku images.

Extracting Cells and Recognizing Characters

With a classifier that recognizes cell contents, the next step is to automatically find the cells. Once we solve that, we can crop them and pass the crops to the classifier. Let's for now assume that the image of the Sudoku is aligned so that the horizontal and vertical lines of the grid are parallel to the image sides (like the left image of Figure 8-8). Under these conditions, we can threshold the image and sum up the pixel values horizontally and vertically. Since the edges will have values of one and the other parts values of zeros, this should give strong response at the edges and tell us where to crop.

The following function takes a grayscale image and a direction and returns the ten edges for that direction:

```
from scipy.ndimage import measurements

def find_sudoku_edges(im,axis=0):
    """ Finds the cell edges for an aligned sudoku image. """

    # threshold and sum rows and columns
    trim = 1*(im<128)
    s = trim.sum(axis=axis)

    # find center of strongest lines
    s_labels,s_nbr = measurements.label(s>(0.5*max(s)))
    m = measurements.center_of_mass(s,s_labels,range(1,s_nbr+1))
    x = [int(x[0]) for x in m]

    # if only the strong lines are detected add lines in between
    if len(x)==4:
        dx = diff(x)
        x = [x[0],x[0]+dx[0]/3,x[0]+2*dx[0]/3,
             x[1],x[1]+dx[1]/3,x[1]+2*dx[1]/3,
             x[2],x[2]+dx[2]/3,x[2]+2*dx[2]/3,x[3]]

    if len(x)==10:
        return x
    else:
        raise RuntimeError('Edges not detected.')
```

First, the image is thresholded at the midpoint to give ones on the dark areas. Then these are added up in the specified direction (axis=0 or 1). The `scipy.ndimage` package contains a module, `measurements`, that is very useful for counting and measuring regions in binary or label arrays. First, `labels()` finds the connected components of a binary array computed by thresholding the sum at the midpoint. Then the `center_of_mass()` function computes the center point of each independent component. Depending on the graphic design of the Sudoku (all lines equally strong or the sub-grid lines stronger than the other), you might get four or ten points. In the case of four, the intermediary lines are interpolated at even intervals. If the end result does not have ten lines, an exception is raised.

In the “sudokus” folder is a collection of Sudoku images of varying difficulty. There is also a file for each image containing the true values of the Sudoku so that we can check